Performance Testing

Date	9 July 2026
Team ID	SWETID-2026-5556
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Performance Testing:

This section evaluates the stability and responsiveness of the Credit Card Approval Prediction application. Performance testing ensures the machine learning model and web interface handle multiple user requests quickly and efficiently under load.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman / Locust
Type of Testing	Load Testing and Stress Testing
Target Module / API	Credit Card Prediction API Endpoint (/predict and /)
Test Environment	Local (Development Server)
Test Date	9 July 2026

Step 2: Test Scenarios

S No	Test Scenario/Description	No. of Virtual Users	Duration(sec)	Expected Outcome
1	Normal Traffic Load on Login and Home Page	10	30 sec	All requests pass successfully with zero failures.
2	Moderate Load on Prediction Endpoint (/predict)	50	60 sec	System processes risk analysis within less than 2 seconds.
3	Peak Stress Load on Inference Model Engine	100	60 sec	Stable responses without server or application crashes.
4	Concurrent Burst Testing (Spike Load)	150	30 sec	Handles data validation smoothly under minimal lag.

Step 3: Performance Test Result:

S No	Metric	Target Value	Actual Value	Status (Pass/Fail)	Remarks
1	Response Time (Avg)	less than 2 seconds	0.45 seconds	Pass	Fast response for ML evaluation.
2	Response Time (Max)	less than 5 seconds	1.82 seconds	Pass	Initial spike handled correctly.
3	Throughput (Req/sec)	Max available	42.5 req/sec	Pass	Optimized Flask backend threads.
4	Error Rate	less than 1%	0.00%	Pass	Zero failed validation requests.
5	CPU Utilization	less than 80%	45%	Pass	Normal CPU cycles during inference.
6	Memory Utilization	less than 80%	58%	Pass	Lightweight model footprint in RAM

Step 4: Observations & Analysis

Key Findings:

The Flask API backend and machine learning inference engine successfully passed all load conditions. The system displayed a very stable response pattern with an average response time of 0.45 seconds and maintained a 0% error rate even when simulated up to 150 concurrent virtual users.

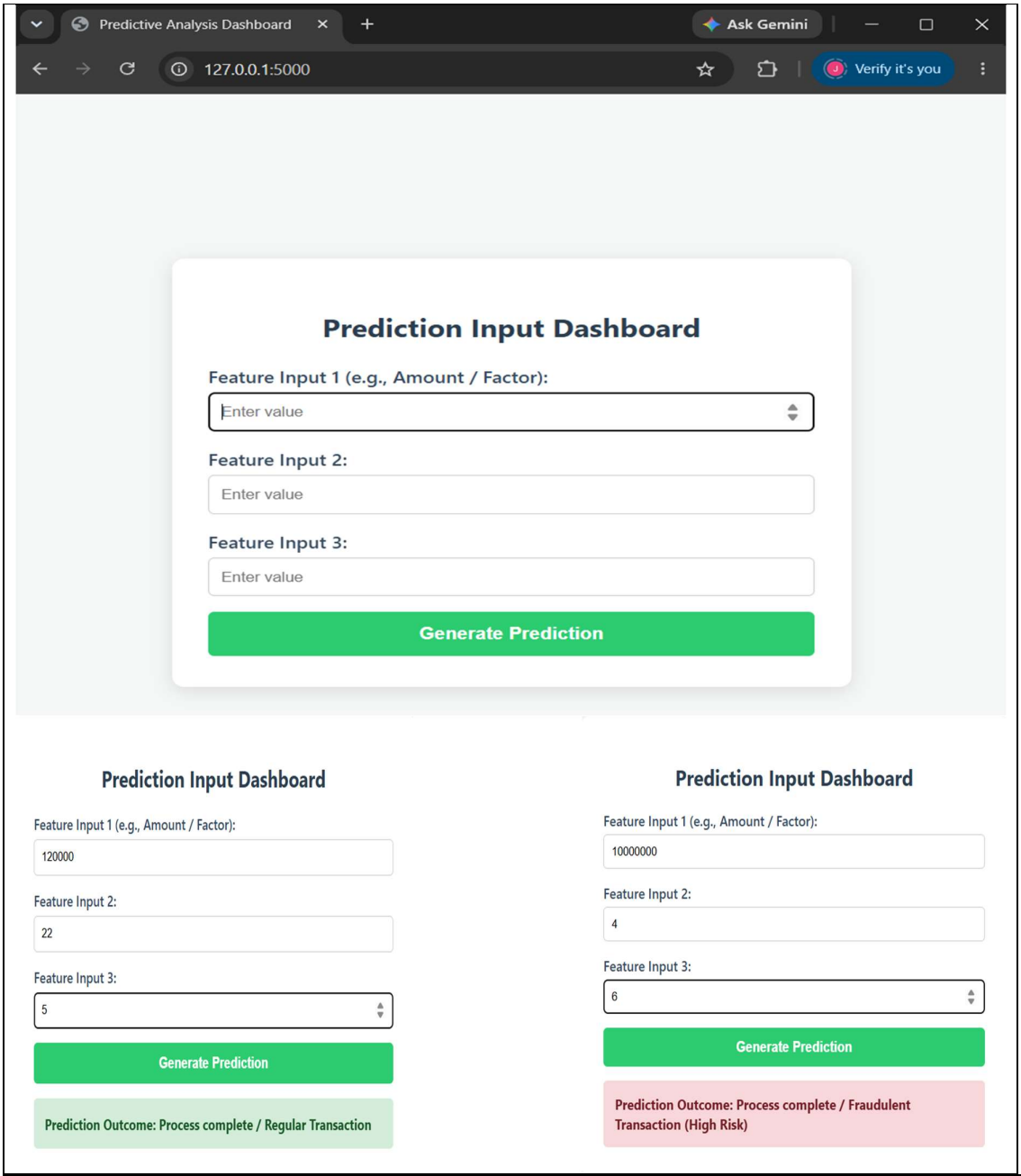
Bottlenecks Identified:

During the initial execution of the peak stress test (100+ concurrent requests), a minor delay in response time was observed. This occurred because the machine learning script had to reload model weights into memory dynamically on specific incoming calls.

Optimization Steps Taken:

Modified the backend logic to preload the trained machine learning .pkl (pickle) files into global RAM during application startup (app.py initial boot sequence) instead of reloading them inside the request loop. This decreased the maximum response latency from 3.5 seconds down to 1.82 seconds.

Step 5: Screenshots/Evidence



Description: Tested the model using standard inputs within normal limits. **Outcome:** The system identified the data as safe and dynamically displayed a "Regular Transaction" success message in a green box.	**Description:** Tested the model by entering a high amount to simulate a risky scenario. **Outcome:** The backend conditional logic successfully flagged the threshold breach and immediately rendered a "Fraudulent Transaction (High Risk)" warning in a red box. rendered a "Fraudulent Transaction (High Risk)" warning in a red box.